

Multi-Tier Computing

Last-Minute Quick Reference

Compiled from course snippets • Units 1 & 2

Snippet 1 #mtc #unit1 — Driving Forces, Issues, Computing Models, C/S Pros & Cons

Driving Forces in Information Technology

- Globalization
- Technological innovation
- Demand for real-time information
- Automation & cost efficiency
- User expectations

Major Issues in Information Technology

- Security threats & data privacy risks
- Integration challenges
- Legacy systems (expensive to modernize)
- Scalability and performance issues
- High IT costs
- Skill gaps

Right Sizing Solution: *“using scalable distributed computing and cloud computing.”*

Computing Models Comparison

Model	Strengths	Weaknesses
Host / Centralised	High reliability	Low flexibility; expensive to scale
Non-Distributed	Easy management	Limited scalability; hard to adapt
Distributed	Scalability, performance, resource sharing, fault tolerance, high availability	Complexity

Decomposition Approaches

- Functional decomposition
- Data decomposition
- Domain-based decomposition
- Object-oriented decomposition

Advantages of Client/Server

- Performance & reduced workload
- System interoperability
- Scalability
- Data integrity
- Resource sharing
- Location independence

Disadvantages of Client/Server

- Maintenance cost
- Training cost
- Hardware cost
- Software cost
- Complexity

Snippet 2 #mtc #unit2 — Client Anatomy: GUI vs. OOUI Clients

GUI Clients

- Occasional requests to server result from a human interacting with a GUI.
- Good fit for mainstream OLTP-type applications.
- Good front-end clients to database servers.

OOUI Clients

- Highly iconic, OO interface — directly manipulate objects via drag-and-drop.
- Used by information workers doing multiple, variable, unpredictable tasks.
- Communications are **real-time, interactive, and highly concurrent**.
- Examples: *Figma*, *Photoshop*, *TradingView*, *Blender*, *Mac OS*, *Windows 98*.

OOUI Desktop Features

- Visual desktop metaphor — bring together related objects to perform a task.
- Multiple concurrent workplaces and parallel sessions.
- Background prefetching of content.
- Provides folders, workareas, and shortcuts.
- Common metaphor for creating, copying, moving, connecting, and deleting objects.
- **Key feature: multiple views of objects.**

OOUI Feature Summary

Feature	Description
Integration / Drag-and-drop	Extension of OS UI; seamlessly integrated. Transactions triggered by dragging objects onto targets.
Notebook View	Icon opens to reveal a notebook that visually staples many dialog windows together.
Persistent Desktop	Desktop configuration reappears exactly as the user left it on restart.
Familiarity	Very intuitive, especially for younger users — feels like a video game.
Extensibility	Seamlessly extended to work with any other OOUI object with very little new code.

Snippet 3 #mtc #unit2 — Modular Programming, OS Services, Client & Server Anatomy

Client/Server as Modular Programming

- Client/Server programming can be viewed as an **extension of Modular Programming**.
- Obsolete modules can be easily upgraded or replaced.
- **P2P**: Each workstation has same responsibilities — poor performance under load.
- **C/S**: Allows single-responsibility servers (file, print, network servers).

Functional Layers of C/S Apps (in order): 1. Presentation Logic / UI 2. Business Logic 3. Data

Tier thinness: 1-Tier < 2-Tier < 3-Tier (increasing thin-ness of client)

OS Services

Basic Services

1. Multitasking / Multithreading
2. Memory management
3. Inter-process communication (IPC)
4. Task prioritization

Extended Services

1. Database management systems (DBMS)
2. Transaction processing monitors (TPMs)
3. Middleware
4. Security services

Anatomy

Client Anatomy

1. User Interface
2. Presentation logic
3. Communication module
4. Local processing
5. Session handling

Server Anatomy

1. Initialization
2. Listening for requests
3. Request handling
4. Business logic
5. Transaction management
6. Response generation

Snippet 4 #mtc #unit1 — Open System Standards

Open system standards allow **heterogeneous clients and servers** (different vendors, OS, hardware) to communicate seamlessly.

Standards Areas

- **Platforms:** Intel Chips, UNIX, POSIX, DOS
- **Networks:** OSI, TCP/IP
- **Middleware**
- **Application**

Why Open Systems?

- Conform to a broad set of formal and de facto standards
- Support for variety of vendors
- Strategic adoption of standards by users and developers
- Plug and play, easier development